معماری مشخصی توصیف می‌شود (برای مثال، یک ABI برای پردازنده‌ی ARMv8 وجود دارد). بنابراین، یک ABI، معادل سطح معماری یک API است. اگر یک فایل اجرایی دودویی کامپایل شده باشد و مطابق یک ABI خاصی متصل شود، بایستی قادر باشد بر روی سیستم‌های متفاوتی که آن ABI را پشتیبانی می‌کنند اجرا شود. با این همه، چون یک ABI به‌خصوص برای سیستم‌عامل خاصی که روی یک معماری مشخصی اجرا می‌شود تعریف می‌شود، ABIها در تدارک سازگاری بین سکویی، کار کمی از پیش می‌برند.

در مجموع، تمامی این اختلاف‌ها به این معنی است که مگر این که یک مفسر، RTE، یا یک فایل اجرایی دودویی نوشته شده و روی یک سیستم‌عامل خاصی بر روی یک نوع پردازنده‌ی خاصی (همچون اینتل x86 یا ARMv8) کامپایل شود، برنامه در اجرا با شکست مواجه خواهد شد. میزان کاری را که برای یک برنامه همچون مرورگر فایرفاکس لازم است تصور کنید تا بر روی ویندوز، macOS، نسخه‌های مختلف لینوکس، iOS، و اندروید، بعضاً هم بر روی معماری‌های مختلف CPU اجرا شود.

۲-۷ طراحی و پیاده‌سازی سیستم‌عامل

در این بخش، مسائلی را بحث می‌کنیم که در طراحی و پیاده‌سازی یک سیستم‌عامل مواجه می‌شویم. البته، راه‌حل کاملی برای این قبیل مسائل وجود ندارد، اما رویکردهایی هستند که موفقیت آنها به اثبات رسیده است.

۲-۷-۱ اهداف طراحی

اولین مسأله در طراحی یک سیستم، تعریف و توصیف اهداف آن است. در بالاترین سطح، طراحی سیستم، تحت تأثیر گزینش سخت‌افزار و نوع سیستم خواهد بود: سیستم رومیزی سابق/لپ‌تاپ، موبایل، توزیع‌شده، یا بلادرنگ.

توصیف نیازمندی‌ها که بالاتر از این سطح طراحی هم قرار دارد می‌تواند بسیار دشوارتر باشد. با این وجود، نیازمندی‌ها می‌توانند به دو گروه اساسی تقسیم شوند که عبارتند از: **اهداف کاربر** و **اهداف سیستم**.

کاربران خصوصیات روشن خاصی از یک سیستم را می‌خواهند. سیستم بایستی کاربرد راحت، یادگیری و استفاده‌ی آسانی داشته، مطمئن، ایمن و سریع باشد. البته، این توصیف‌ها مخصوصاً در طراحی سیستم سودمند نیستند، چرا که هیچ‌گونه توافق کلی برای حصول این اهداف وجود ندارد.

مجموعه‌ی مشابهی از نیازمندی‌ها هم می‌تواند توسط توسعه‌دهندگانی که می‌بایست سیستم را طراحی، ایجاد، نگه‌داری و عملیاتی کنند، تعریف شود. سیستم بایستی طراحی، پیاده‌سازی و نگه‌داری آسانی داشته و انعطاف‌پذیر، مطمئن، عاری از خطا و کارآمد باشد. باز هم این نیازمندی‌ها مبهم هستند و به شیوه‌های مختلفی قابل تفسیرند.

به طور خلاصه، هیچ راه‌حل واحدی برای مسأله‌ی تعریف نیازمندی‌های یک سیستم‌عامل وجود ندارد. گستردگی سیستم‌های موجود نشان می‌دهد که نیازمندی‌های متفاوت می‌توانند تنوع

گسترده‌ای را برای راه‌حل‌های مختلف محیط‌های متفاوت به دنبال داشته باشند. برای مثال، نیازمندی‌های سیستم‌عامل بلادرنگ Wind River VxWorks سیستم‌های تعبیه شده بایستی با نیازمندی‌های Windows Server که یک سیستم‌عامل چنددسترسی بزرگ برای کاربردهای کلان‌شرکتی طراحی شده است، تفاوت‌های قابل توجهی داشته باشد.

توصیف و طراحی سیستم‌عامل، کاری است که خلاقیت بالایی را می‌طلبد. اگرچه هیچ کتابی نمی‌تواند به شما بگوید که چگونه این کار را انجام بدهید، یک سری اصول کلی در شاخه‌ی **مهندسی نرم‌افزار** توسعه داده شده است که برخی از این اصول را در ادامه بحث خواهیم کرد.

۲-۷-۲ راهکارها و سیاست‌ها

یک اصل مهم، تفکیک **سیاست** از **راهکار**[1] است. راهکارها چگونگی انجام کاری را تعیین می‌کنند؛ سیاست‌ها *آنچه* را که انجام خواهد گرفت، مشخص می‌کنند. برای مثال، بنای تایمر (بخش ۱-۴-۳ را ببینید)، راهکاری برای تضمین حفاظت پردازنده است، اما تصمیم این که تایمر به چه مدت برای یک کاربر خاص انتخاب شود، اتخاذ سیاست است.

تفکیک سیاست و راهکار برای داشتن انعطاف‌پذیری اهمیت دارد. سیاست‌ها به احتمال فراوان در مکان‌ها یا زمان‌های مختلف تغییر خواهند کرد. در بدترین حالت، هرگونه تغییر در سیاست مستلزم یک تغییر در راهکار زیرین آن خواهد بود. یک راهکار کلی که با انعطاف کافی برای کار در میان بازه‌ای از سیاست‌ها، مطلوبیت بیشتری خواهد داشت. در این صورت، یک تغییر در سیاست، تنها به تعریف مجدد پارامترهای خاصی از سیستم نیاز دارد. برای نمونه، راهکاری را در نظر بگیرید که به انواع خاصی از برنامه‌ها نسبت به سایرین الویت می‌دهد. اگر راهکار به درستی از سیاست تفکیک شده باشد، می‌توان سیاستی اتخاذ کرد که برنامه‌های پاپیچ[2] I/O را نسبت به برنامه‌های پاپیچ پردازنده الویت بدهد و یا عکس این سیاست اعمال شود.

سیستم‌عامل‌های مبتنی بر ریزهسته[3] (که در بخش ۲-۸-۳ بحث می‌شوند)، تفکیک راهکار و سیاست را با پیاده‌سازی یک مجموعه‌ی پایه از بلاک‌های سازنده‌ی ابتدایی، به نهایت حد خود می‌رسانند. این بلاک‌ها تقریباً رها از سیاست هستند که امکان افزودن راهکارها و سیاست‌های پیشرفته‌تر را از طریق ماژول‌های هسته که کاربر ایجاد کرده یا از طریق خود برنامه‌های کاربر، فراهم می‌کنند. در مقایسه، ویندوز را در نظر بگیرید، سیستم‌عامل تجاری بسیار فراگیری که بیش از سه دهه است در دسترس می‌باشد. مایکروسافت به‌صورت تنگاتنگ، توأمان راهکار و سیاست را در سیستم کد کرده است تا یک ظاهر و حس سراسری در میان تمامی دستگاه‌هایی که سیستم‌عامل ویندوز را اجرا می‌کنند به‌وجود بیاورد. تمامی برنامه‌های کاربردی رابط‌های مشابهی دارند، چون خود رابط در داخل هسته و کتابخانه‌های سیستمی ساخته شده است. اَپل یک راهبرد مشابهی را در سیستم‌عامل‌های macOS و iOS تطبیق داده است.

1. Mechanism
2. Intensive
3. Microkernel-based

می‌توانیم یک مقایسه‌ی مشابهی میان سیستم‌عامل‌های تجاری و متن‌باز انجام دهیم. برای نمونه، ویندوز را که در بالا بحث شد با لینوکس، یک سیستم‌عامل متن‌باز مقایسه کنید که بر روی بازه‌ی وسیعی از دستگاه‌های محاسباتی اجرا می‌شود و بیش از ۲۵ سال است که در دسترس می‌باشد. هسته‌ی «استاندارد» لینوکس یک الگوریتم زمان‌بندی CPU خاصی دارد (در بخش ۵-۷-۱ پوشش داده می‌شود) که راهکاری است که از یک سیاست خاصی پشتیبانی می‌کند. با این وجود، هرکس آزاد است تا زمان‌بند را برای پشتیبانی یک سیاست متفاوتی اصلاح کرده یا جایگزین نماید.

اتخاذ سیاست‌ها برای تمامی تخصیص منبع اهمیت دارد. هر بار که لازم است تصمیم‌گیری شود آیا منبعی تخصیص یابد یا خیر، بایستی یک سیاست اتخاذ شود. هر بار که سؤال چگونه به جای چیستی مطرح است، یک راهکار است که بایستی تعیین شود.

۲-۷-۳ پیاده‌سازی

همین که سیستم‌عامل طراحی شد، بایستی پیاده‌سازی شود. چون سیستم‌عامل‌ها مجموعه‌هایی از برنامه‌های بسیاری هستند که توسط افراد زیادی در طی یک دوره‌ی زمانی طولانی نوشته می‌شود، تعابیر کلی در رابطه با چگونگی پیاده‌سازی آنها دشوار است.

سیستم‌عامل‌های اولیه به زبان اسمبلی نوشته می‌شدند. اکنون، اکثراً به زبان‌های سطح بالایی همچون C یا C++ با مقدار کوچکی که از سیستم که به‌زبان اسمبلی نوشته می‌شوند. به واقع، اغلب بیش از یک زبان سطح بالاتر استفاده می‌شود. پایین‌ترین سطوح هسته ممکن است به زبان اسمبلی و C نوشته شوند. روال‌های سطح بالاتر به زبان C و C++، و کتابخانه‌های سیستمی ممکن است به زبان C++ یا حتی زبان‌های سطح بالاتر نوشته شوند. اندروید یک مثال عالی ارائه می‌دهد: هسته‌اش عمدتاً با C نوشته شده با کمی هم زبان اسمبلی. اکثر کتابخانه‌های سیستم اندروید به‌زبان C یا C++ نوشته شده، و چارچوب‌های برنامه‌های کاربردی آن—که رابط برنامه‌نویس به سیستم را فراهم می‌کنند—عمدتاً به‌زبان جاوا نوشته شده‌اند. ما معماری اندروید را با جزئیات بیشتر در بخش ۲-۸-۵-۲ پوشش می‌دهیم.

مزایایی که در استفاده از یک زبان سطح بالاتر یا حداقل در زبان پیاده‌سازی سیستم‌ها برای پیاده‌سازی سیستم‌عامل‌ها وجود دارد همچون مواردی است که در برنامه‌های کاربردی حاصل می‌شود: کدنویسی سریع‌تر انجام می‌شود، فشرده‌تر است و درک و عیب‌یابی آن آسان‌تر است. به علاوه، پیشرفت‌های فن‌آوری کامپایلر، کد تولیدی کل سیستم‌عامل را به سادگی با کامپایل مجدد بهبود خواهد بخشید. بالاخره، اگر سیستم‌عامل به زبان سطح بالاتری نوشته شده باشد، **قابلیت حمل** آسانی در انتقال به سخت‌افزار دیگر خواهد داشت. این موضوع مخصوصاً برای سیستم‌عامل‌هایی که دنبال این هستند که بر روی چندین سیستم سخت‌افزاری متفاوت، همچون دستگاه‌های تعبیه‌شده‌ی کوچک، سیستم های x86 اینتل، و تراشه‌های ARM که روی گوشی‌ها و تبلت‌ها اجرا می‌شوند، اهمیت دارد.

تنها معایبی که ممکن است در پیاده‌سازی یک سیستم‌عامل به زبان سطح بالاتر وجود داشته باشند، کاهش سرعت و ازدیاد نیازهای حافظه هستند. با این وجود، این مورد در سیستم‌های

امروزی دغدغه‌ی مهمی نیست. هرچند که یک برنامه‌نویس زبان اسمبلی خبره می‌تواند روال‌های کوچک کارآمدی تولید کند، اما یک کامپایلر پیشرفته برای برنامه‌های بزرگ می‌تواند تحلیل و بهینه‌سازی‌های پیچیده‌ای اعمال کند که حاصل آن کد ممتازی خواهد بود. پردازنده‌های مدرن با داشتن خطلوله‌های عمیق[1] و واحدهای کارکرد چندگانه می‌توانند بسیار آسان‌تر از توانایی ذهن بشر با جزئیات وابستگی‌های پیچیده کنار بیایند.

همان‌گونه که در سایر سیستم‌ها نیز صادق است، بهبود اساسی کارایی سیستم‌عامل‌ها می‌تواند با استفاده از الگوریتم‌ها و ساختمان‌داده‌های بهتر حاصل شود تا با کد زبان اسمبلی ممتاز. به علاوه، به رغم این که سیستم‌عامل‌ها بزرگ هستند، تنها بخش کوچکی از کد آن برای داشتن کارایی بالا حساسیت دارد؛ راه‌اندازهای وقفه، مدیر I/O، مدیر حافظه و زمانبند پردازنده احتمالاً روال‌هایی با حساسیت بیشتر خواهند داشت. بعد از این که سیستم نوشته شد و به درستی کار کرد، گلوگاه‌ها می‌تواند شناسایی شده و برای عملیات کارآمدتر به‌سازی[2] شود.

۲-۸ ساختار سیستم‌عامل

اگر قرار باشد که سیستمی به بزرگی و پیچیدگی سیستم‌عامل مدرن به درستی کار کرده و به سادگی هم قابل تغییر باشد بایستی به دقت مهندسی شود. روش مرسوم آن است که این کار به جای یک سیستم منفرد به مؤلفه‌های کوچک‌تر، یا ماژول‌هایی تقسیم شود. هر یک از این ماژول‌ها بایستی به عنوان بخشی از سیستم، با تعریف دقیق رابط‌ها و کارکردها به خوبی تعریف شوند. شما ممکن است موقع ساختاردهی برنامه‌های خودتان از رویکرد مشابهی استفاده کنید: به‌جای این که تمامی کد خود را در تابع main() بگذارید، درعوض منطق را به تعدادی تابع تفکیک کرده، پارامترها و مقادیر برگشتی را به‌وضوع مشخص کرده و سپس، توابع را از main() فراخوانی کنید.

مؤلفه‌های رایج سیستم‌عامل‌ها را به اختصار در فصل ۱ بحث کردیم. در این بخش، چگونگی اتصال این مؤلفه‌ها به همدیگر و ترکیب آنها را با هسته بحث می‌کنیم.

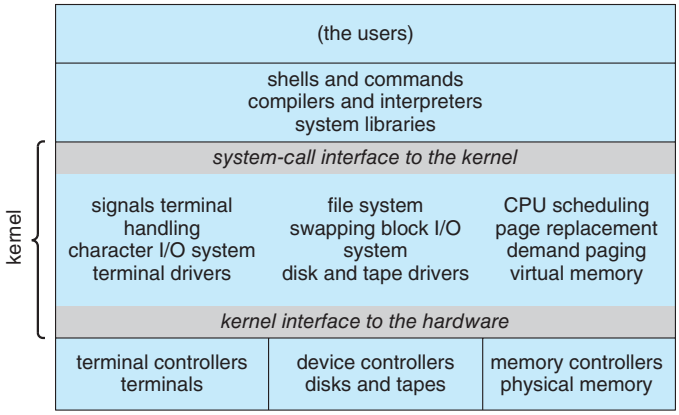
۲-۸-۱ ساختار یک‌تکه

ساده‌ترین ساختار برای سازمان‌دهی یک سیستم‌عامل، اصلاً نداشتن هیچ ساختار می‌باشد. یعنی، تمامی کارکرد هسته در یک فایل منفرد ایستا گذاشته شود که در یک فضای آدرس منفردی اجرا می‌شود. این رویکرد—تحت عنوان ساختار **یک‌تکه**[3]—تکنیک مرسومی برای طراحی سیستم‌عامل‌هاست.

مثالی از چنین ساختار محدود شده‌ای، سیستم‌عامل یونیکس اولیه است که شامل دو بخش مجزاست: هسته و برنامه‌های سیستمی. هسته خود به یک سری رابط و درایور دستگاه تفکیک می‌شد که در طول سال‌ها با تکامل یونیکس افزوده شده و گسترش پیدا کرده‌اند. می‌توانیم

1. Deep pipelining
2. Refactor
3. Monolithic

سیستم‌عامل یونیکس مرسوم را که تا حدودی لایه‌دار است، به صورت شکل ۲-۱۲ ببینیم. هر چیزی که زیر رابط فراخوانی سیستمی و بالای سخت‌افزار فیزیکی قرار دارد، هسته است. هسته سیستم‌فایل، زمانبندی پردازنده، مدیریت حافظه و سایر کارکردهای سیستم‌عامل را از طریق فراخوانی‌های سیستمی فراهم می‌کند. در مجموع، حجم عظیمی از کارکردها در یک فضای آدرس منفردی ترکیب می‌شود.



شکل ۲-۱۲ ساختار سیستم یونیکس سابق.

سیستم‌عامل لینوکس مبتنی بر یونیکس می‌باشد و مشابه آن ساختاردهی می‌شود، آن‌گونه که در شکل ۲-۱۳ نشان داده می‌شود. برنامه‌های کاربردی در زمان ارتباط با رابط فراخوانی سیستمی به هسته نوعاً از کتابخانه‌ی استاندارد glibc زبان C استفاده می‌کنند. هسته‌ی لینوکس یک‌تکه است از این نظر که به‌صورت کامل در مد هسته در یک فضای آدرس منفردی اجرا می‌شود، اما همان‌گونه که در بخش ۲-۸-۴ خواهیم دید، یک طراحی ماژولار دارد که به هسته اجازه می‌دهد تا در طی زمان اجرا تغییر داده شود.

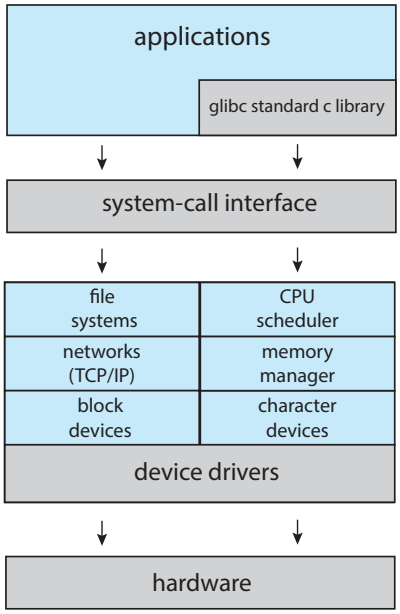
به‌رغم سادگی ظاهری هسته‌های یک‌تکه، پیاده‌سازی و گسترش دشواری دارند. با این وجود، هسته‌های یک‌تکه یک برتری کارایی قابل توجهی دارند: سربار بسیار کوچکی در رابط فراخوانی سیستمی وجود دارد، و ارتباط در میان هسته سریع می‌باشد. بنابراین، به‌رغم معایب و موانع هسته‌های یک‌تکه، سرعت و کارآمدی آن‌هاست که مشخص می‌کند چرا ما همچنان شاهد این ساختار در سیستم‌عامل‌های یونیکس، لینوکس، و ویندوز هستیم.

۲-۸-۲ رویکرد لایه‌لایه

رویکرد یک‌تکه غالباً تحت عنوان سیستم **با تزویج سخت**[1] شناخته می‌شود چون تغییرات به یک بخش از سیستم می‌تواند بازه‌ی اثرات وسیعی بر روی سایر بخش‌ها داشته باشد. در روشی دیگر، می‌توانیم یک سیستمی **با تزویج نرم**[2] طراحی کنیم. چنین سیستمی به مؤلفه‌های

1. Tightly coupled
2. Loosely coupled

جداگانه‌ی کوچک‌تری تقسیم می‌شود که کارکرد خاص و محدود شده‌ای دارند. تمامی این مؤلفه‌ها با همدیگر هسته را تشکیل می‌دهند. حسن این رویکرد ماژولار این است که تغییرات در یک مؤلفه تنها آن مؤلفه را متأثر می‌سازند، و دیگر هیچ، این موضوع به پیاده‌سازان سیستم آزادی عمل بیشتری در ایجاد و تغییر کارهای داخلی سیستم می‌دهد.



۲-۱۳ ساختار سیستم لینوکس

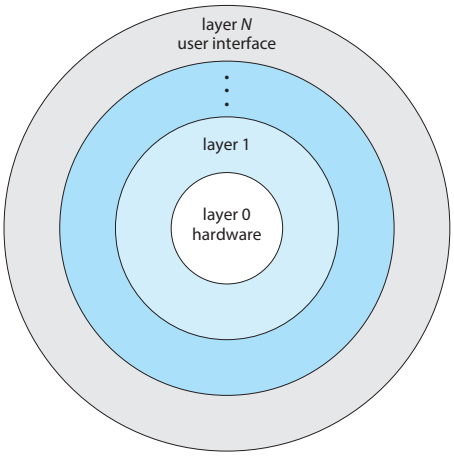
یک سیستم می‌تواند به روش‌های زیادی ماژولار ساخته شود. یک متد این کار، **رویکرد لایه‌لایه**[1] است که در آن، سیستم‌عامل به تعدادی لایه (سطح) شکسته می‌شود. پایین‌ترین لایه (لایه‌ی ۰) سخت‌افزار است؛ بالاترین لایه (لایه‌ی N)، رابط کاربر است. این ساختار لایه‌بندی[2] در شکل ۲-۱۴ به تصویر کشیده می‌شود.

هر لایه‌ی سیستم‌عامل، پیاده‌سازی یک شئ انتزاعی است که از داده‌ها و عملیاتی تشکیل می‌شود که آن داده‌ها را دستکاری می‌کنند. یک لایه‌ی نوعی سیستم‌عامل—بر فرض، لایه‌ی M—شامل ساختمان‌داده‌ها و مجموعه‌ای از کارکردهاست که می‌توانند توسط لایه‌های سطح بالاتر احضار شوند. لایه‌ی M هم به نوبت خود می‌تواند عملیات لایه‌های سطح پایین‌تر را احضار کند.

مزیت اصلی رویکرد لایه‌لایه، سادگی ساخت و عیب‌یابی آن است. گزینش لایه‌ها طوری است که هر یک از آنها فقط از توابع (عملیات) و سرویس‌های لایه‌های سطح پایین‌تر استفاده می‌کنند. این رویکرد، عیب‌یابی و وارسی[3] سیستم را ساده‌تر می‌کند. اولین لایه بی‌آنکه نگران

1. Layered approach
2. Layering
3. Verification

بقیه‌ی سیستم باشد عیب‌یابی می‌شود، چون طبق تعریف، برای پیاده‌سازی کارکردهای خود فقط از سخت‌افزار پایه (که درست فرض می‌شود) استفاده می‌کند. همین که اولین لایه عیب‌یابی شد، می‌توان در زمان خطایابی لایه‌ی دوم، کارکرد آن را درست فرض کرد و الی آخر. اگر طی عیب‌یابی لایه‌ای خاص، خطایی پیدا شود، بایستی خطا در همان لایه اتفاق افتاده باشد، چون لایه‌های زیرین آن لایه پیشتر عیب‌یابی شده‌اند. بنابراین، طراحی و پیاده‌سازی سیستم آسان می‌شود.



شکل ۲-۱۴ یک سیستم‌عامل لایه‌لایه.

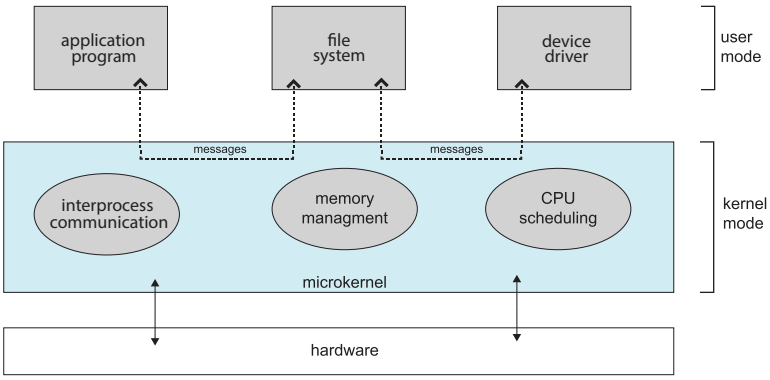
هر لایه فقط با استفاده از آن دسته از عملیاتی که توسط لایه‌های سطح پایین‌تر فراهم می‌شوند، پیاده‌سازی می‌شود. یک لایه نیازی ندارد که از چگونگی پیاده‌سازی این عملیات آگاهی داشته باشد؛ تنها لازم دارد که بداند این عملیات چه کاری را انجام می‌دهند. از این رو، هر لایه ساختمان‌داده‌ها، عملیات و سخت‌افزار خاص موجود را از لایه‌های سطح بالاتر پنهان می‌کند.

سیستم‌های لایه‌لایه به‌صورت موفقیت‌آمیز در شبکه‌های کامپیوتری (همچون TCP/IP) و برنامه‌های کاربردی وب استفاده شده‌اند. با این همه، سیستم‌عامل‌های نسبتاً کمی از رویکرد لایه‌لایه‌ی محض استفاده می‌کنند. یک دلیل آن به چالش‌های تعریف مناسب کارکرد هر لایه بر می‌گردد. به‌علاوه، کارایی کلی این قبیل سیستم‌ها به‌واسطه‌ی سربار ضرورت پیمایش برنامه‌ی کاربر در میان لایه‌ها به‌منظور دست‌یابی به یک سرویس سیستم‌عامل ضعیف است. با این وجود، یک مقدار لایه‌گذاری در سیستم‌عامل‌های کنونی مرسوم است. در حالت کلی، این سیستم‌ها لایه‌های کمتری با کارکرد بیشتری دارند تا مزایای بیشتری از کد ماژولار فراهم شود، در عین حال از مسائل تعریف و تعامل لایه اجتناب گردد.

۲-۸-۳ ریزهسته‌ها

پیش از این دیدیم که سیستم یونیکس اولیه یک ساختار یک‌تکه داشت. همچنان‌که یونیکس گسترش پیدا کرد، هسته بزرگ شده و مدیریت آن دشوار شد. در اواسط دهه‌ی ۱۹۸۰، محققان

دانشگاه کارنگی ملون[1] سیستم‌عاملی به نام **Mach** را توسعه دادند که هسته را با استفاده از یک رویکرد **ریزهسته** ماژولار می‌ساخت. این رویکرد، سیستم‌عامل را با برداشتن تمامی مؤلفه‌های غیرضروری هسته و پیاده‌سازی آنها به صورت برنامه‌های سطح کاربر که در فضاهای آدرس مجزایی مقیم هستند، ساخت‌یافته‌تر می‌کند. حاصل، یک هسته‌ی کوچک‌تر است. اجماع کمی در رابطه با این که چه سرویس‌هایی در هسته باقی بمانند و کدام یک از آنها در فضای کاربر پیاده‌سازی شوند وجود دارد. با این همه، ریزهسته‌ها نوعاً علاوه بر یک امکان ارتباط، حداقل مدیریت پردازه و حافظه را فراهم می‌کنند. شکل ۲-۱۵، معماری یک ریزهسته‌ی نوعی را به تصویر می‌کشد.



شکل ۲-۱۵ معماری یک ریزهسته‌ی نوعی.

هدف اصلی ریزهسته فراهم‌ساختن ارتباط میان برنامه‌ی کارفرما و سرویس‌های مختلفی است که آنها هم در فضای کاربر اجرا می‌شوند. ارتباط از طریق **عبور پیغام** فراهم می‌شود که در بخش ۲-۳-۳-۵ مطرح شد. برای مثال، اگر برنامه‌ی کارفرما بخواهد به یک فایل دسترسی پیدا کند، بایستی با کارگزار فایل تعامل داشته باشد. برنامه‌ی کارفرما و سرویس، هرگز تعامل مستقیم ندارند. بلکه، به صورت غیر مستقیم از طریق تبادل پیغام با ریزهسته ارتباط برقرار می‌کنند.

یکی از منافع رویکرد ریزهسته این است که توسعه‌ی سیستم‌عامل را تسهیل می‌بخشد. تمامی سرویس‌های جدید به فضای کاربر اضافه شده و در نتیجه تغییر هسته را لازم ندارند. زمانی که قرار است هسته به ناچار تغییر یابد، این تغییرات بسیار اندک است، چرا که ریزهسته هسته‌ای کوچک‌تر است. سیستم‌عامل حاصل نیز به راحتی از یک طرح سخت‌افزاری به دیگری قابل انتقال است. ریزهسته، امنیت و قابلیت اطمینان بیشتری هم فراهم می‌کند، چون بیشتر سرویس‌ها به عنوان پردازه‌های کاربر—به جای هسته—اجرا می‌شوند. اگر سرویسی از کار بیفتد، باقی سیستم‌عامل دست‌نخورده باقی می‌ماند.

1. Carnegie Mellon University

شاید بهترین نمود یک سیستم‌عامل ریزهسته، *Darwin* باشد، مؤلفه‌ی هسته‌ی سیستم‌عامل‌های macOS و iOS. Darwin در حقیقت، شامل دو هسته است، یکی از آن‌ها ریزهسته‌ی Mach می‌باشد. ما سیستم‌های macOS و iOS را با جزئیات بیشتری در بخش ۲-۸-۵-۱ پوشش خواهیم داد.

مثال دیگر QNX، سیستم‌عامل بلادرنگ برای سیستم‌های تعبیه شده است. ریزهسته‌ی QNX Neutrino، سرویس‌هایی برای عبور پیغام و زمانبندی پردازه فراهم می‌کند. همین‌طور، ارتباطات سطح پایین شبکه و وقفه‌های سخت‌افزاری را راه‌اندازی می‌کند. سایر سرویس‌های QNX، همگی توسط پردازه‌های استانداری فراهم می‌شوند که خارج از هسته در مد کاربر اجرا می‌شوند.

متأسفانه، کارایی ریزهسته‌ها می‌تواند به دلیل افزایش سربار کارکرد سیستم تقلیل یابد. زمانی که دو سرویس سطح کاربر می‌بایست ارتباط برقرار کنند، بایستی پیغام‌ها میان سرویس‌ها کپی شوند، که در فضاهای آدرس جداگانه‌ای قرار دارند. به‌علاوه، سیستم‌عامل امکان دارد که مجبور شود از یک پردازه به دیگری سوییچ کند تا پیغام‌ها را مبادله نماید. سربار مشمول کپی‌کردن پیغام‌ها و سوییچ میان پردازه‌ها، بزرگترین مانع رشد سیستم‌عامل‌های ریزهسته بوده است. تاریخچه‌ی ویندوز NT را در نظر بگیرید: اولین نسخه، یک سازمان ریزهسته‌ی لایه‌لایه داشت. کارایی این نسخه در مقایسه با ویندوز ۹۵ پایین بود. ویندوز NT 4.0 با انتقال لایه‌ها از فضای کاربر به فضای هسته و مجتمع‌سازی نزدیک آنها، کارایی را تا حدودی بهبود بخشید. زمانی که ویندوز XP طراحی شد، بیشتر معماری ویندوز به جای ریزهسته، یک‌تکه شد. بخش ۲-۸-۵-۱ بیان خواهد کرد که چگونه macOS به ملاحظات کارایی ریزهسته‌ی Mach می‌پردازد.

۲-۸-۴ ماژول‌ها

شاید بهترین متدلوژی حال حاضر طراحی سیستم‌عامل، استفاده از **ماژول‌های هسته‌ی بارشدنی** است. در اینجا، هسته یک مجموعه مؤلفه‌ی اصلی دارد و می‌تواند سرویس‌های اضافه را نیز از طریق ماژول‌ها، در زمان بوت و یا در طی زمان اجرا متصل کند. این نوع طراحی در پیاده‌سازی‌های مدرن یونیکس، همچون لینوکس، macOS، و سولاریس، همین‌طور ویندوز رایج است.

ایده‌ی طراحی این است که هسته سرویس‌های اصلی را فراهم کند، در حالی که سرویس‌های دیگر به صورت پویا در زمان اجرای هسته پیاده‌سازی می‌شوند. اتصال پویای سرویس‌ها به افزودن مستقیم امکانات جدید به هسته ارجحیت دارد چرا که مستلزم کامپایل مجدد هسته با هر بار تغییر است. بنابراین، برای مثال، ممکن است الگوریتم‌های زمانبندی پردازنده و مدیریت حافظه را مستقیم در میان هسته بسازیم و سپس پشتیبانی سیستم‌فایل‌های متفاوت را به شیوه‌ی ماژول‌های بارشدنی اضافه کنیم.

نتیجه‌ی کلی به یک سیستم لایه‌لایه می‌ماند که در آن، هر بخش هسته رابط‌های تعریف شده و حفاظت شده دارد؛ اما از یک سیستم لایه‌لایه انعطاف‌پذیرتر است، چرا که هر ماژولی می‌تواند ماژول‌های دیگر را صدا بزند. این رویکرد، شبیه رویکرد ریزهسته نیز هست که در آن، ماژول اصلی فقط توابع اصلی و دانش چگونگی بارگذاری و ارتباط با سایر ماژول‌ها را دارد؛ اما کارآمدتر است، چون ماژول‌ها به منظور برقراری ارتباط نیازی به احضار عبور پیغام ندارند.

لینوکس اصولاً از ماژول‌های هسته‌ی بارشدنی برای پشتیبانی درایور دستگاه‌ها و سیستم‌فایل‌ها استفاده می‌کند. LKMها می‌توانند با شروع به‌کار سیستم (یا *بوت شدن*) یا در طی زمان اجرا، مثلاً زمانی که یک دستگاه USB به یک ماشین در حال اجرا متصل می‌شود، در هسته «درج» شوند. اگر هسته‌ی لینوکس درایور لازم را نداشته باشد، آن می‌تواند به‌صورت پویا بارگذاری شود. LKMها می‌توانند در طی زمان اجرا نیز از هسته برداشته شوند. LKMها برای لینوکس، امکان داشتن هسته‌ای پویا و ماژولار را فراهم می‌کنند، در حالی که منافع کارایی یک سیستم یک‌تکه را حفظ می‌کنند. ما ایجاد LKMها را در لینوکس در چندین تمرین برنامه‌نویسی در انتهای همین فصل پوشش می‌دهیم.

۲-۸-۵ سیستم‌های ترکیبی

در عمل، سیستم‌عامل‌های بسیار کمی از یک ساختار منفرد که تعریف محضی دارد، پیروی می‌کنند. بلکه، ساختارهای متفاوت را ترکیب می‌کنند که حاصل ترکیب سیستم‌هایی است که ملاحظات کارایی، امنیتی و قابلیت استفاده را مد نظر دارد. برای مثال، لینوکس یک‌تکه است، چون داشتن سیستم‌عامل در یک فضای آدرس منفرد، کارایی بسیار کارآمدی فراهم می‌کند. با این همه، ماژولار هم هست، لذا کارکرد جدید می‌تواند به صورت پویا به هسته افزوده شود. ویندوز نیز به صورت وسیعی یک‌تکه است (باز هم به دلایل کارایی)، اما برخی رفتار نوعی ریزهسته را شامل فراهم‌ساختن پشتیبانی زیرسیستم‌های مجزا (تحت عنوان *رفتارهای*[1] سیستم‌عامل) نگه می‌دارد که به صورت پردازه‌های مد کاربر اجرا می‌شوند. سیستم‌های ویندوز از ماژول‌های هسته‌ی بارشدنی پویا نیز پشتیبانی می‌کنند. کاربردهای عملی لینوکس و ویندوز ۱۰ را به‌ترتیب در فصل ۲۰ و فصل ۲۱ تدارک دیده‌ایم. در باقی این بخش، ساختار سه سیستم ترکیبی را بررسی می‌کنیم که عبارتند از: سیستم‌عامل macOS اَپل و دو سیستم‌عامل فراگیر موبایل—iOS و اندروید.

۲-۸-۵-۱ macOS و iOS

سیستم‌عامل macOS اَپل طراحی شده است تا اصولاً بر روی سیستم‌های کامپیوتری رومیزی و لپ‌تاپ اجرا شود، در حالی که iOS یک سیستم‌عامل موبایل است که برای گوشی هوشمند iPhone و کامپیوتر تبلت iPad طراحی شده است. از نظر معماری، macOS و iOS موارد مشترکی دارند، و لذا ما آن‌ها را با همدیگر ضمن برجسته‌ساختن موارد مشترک، همین‌طور تفاوت آن‌ها ارائه می‌کنیم. معماری کلی این دو سیستم در شکل ۲-۱۶ نشان داده می‌شود. نکات برجسته‌ی لایه‌های مختلف به‌صورت زیر قابل ذکر هستند:

- **لایه‌ی ارتباط کاربر:** این لایه یک رابط نرم‌افزاری تعریف می‌کند که به‌کاربران امکان می‌دهد تا با دستگاه‌های محاسباتی تعامل برقرار کنند. macOS از رابط کاربر **Aqua** استفاده می‌کند که برای یک ماوس یا trackpad طراحی شده است، در حالی که iOS از

1. Personalities

رابط کاربر **Springboard** استفاده می‌کند که برای دستگاه‌های لمسی طراحی شده است.

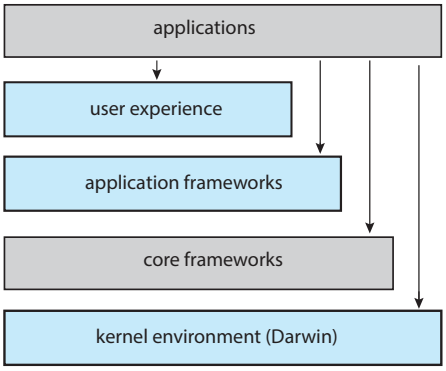
- **لایه‌ی چارچوب‌های برنامه‌ی کاربردی**[1]: این لایه شامل چارچوب‌های **Cocoa** و **Cocoa Touch** می‌باشد که یک API برای زبان‌های برنامه‌نویسی Objective-C و Swift فراهم می‌کند. تفاوت اصلی میان Cocoa و Cocoa Touch در این است که اولی برای برنامه‌نویسی برنامه‌های کاربردی macOS استفاده می‌شود، و دومی توسط iOS برای تدارک پشتیبانی امکانات سخت‌افزاری خاص دستگاه‌های موبایل، همچون صفحات نمایش لمسی استفاده می‌شود.
- **چارچوب‌های اصلی**[2]: این لایه چارچوب‌هایی را تعریف می‌کند که از گرافیک و چندرسانه‌ای اعم از Quicktime و OpenGL پشتیبانی می‌کنند.
- **محیط هسته:** این محیط که به **Darwin** نیز شهرت دارد شامل ریزهسته‌ی Mach و هسته‌ی یونیکس BSD می‌باشد. ما به‌اختصار Darwin را تشریح خواهیم کرد.

همان‌گونه که در شکل ۲-۱۶ نشان داده می‌شود، برنامه‌های کاربردی می‌توانند طوری طراحی شوند تا از امکانات رابط کاربر بهره‌برداری کرده یا آن‌ها را دور زده و مستقیماً با چارچوب برنامه‌ی کاربردی و یا چارچوب اصلی تعامل برقرار کنند. به‌علاوه، یک برنامه‌ی کاربردی می‌تواند چارچوب‌ها را کاملاً صرف‌نظر کرده و مستقیماً با محیط هسته تعامل برقرار کند. (مثالی از این مورد دومی، یک برنامه‌ی C می‌باشد که بدون هیچ رابط کاربری نوشته شده است که فراخوانی‌های سیستمی POSIX را انجام می‌دهد.)

برخی تفاوت‌های قابل توجه میان macOS و iOS شامل موارد زیر است:

- چون macOS با هدف سیستم‌های کامپیوتری رومیزی و لپ‌تاپ طراحی شده است، کامپایل می‌شود تا بر روی معماری‌های اینتل اجرا داشته باشد. iOS برای دستگاه‌های موبایل طراحی شده و لذا برای معماری‌های مبتنی بر ARM کامپایل می‌شود. با تشابهی، هسته‌ی iOS اصلاح شده است تا اندازه‌ای، امکانات خاص و نیازهای سیستم‌های موبایل را مرتفع سازد، همچون مدیریت برق و مدیریت حافظه‌ی پرتکاپو[3]. به‌علاوه، برنامه‌های امنیتی iOS قوی‌تر از macOS است.
- سیستم‌عامل iOS عموماً برای برنامه‌نویسان محدودتر از macOS است و حتی ممکن است به برنامه‌نویسان نزدیک‌تر باشد. برای مثال، iOS دسترسی به APIهای POSIX و BSD را بر روی iOS محدود می‌کند، در حالی که به‌صورت آزاد در دسترس برنامه‌نویسان macOS هستند.

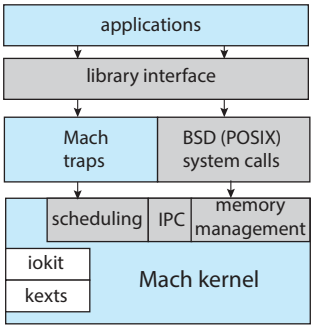
1. Application frameworks layer
2. Core frameworks
3. Aggressive



شکل ۲-۱۶ معماری سیستم‌عامل‌های macOS و iOS اَپل

اکنون بر روی Darwin تمرکز می‌کنیم که از یک ساختار ترکیبی استفاده می‌کند. Darwin یک سیستم لایه‌لایه است که اساساً شامل ریزهسته‌ی Mach و هسته‌ی یونیکس BSD می‌باشد. ساختار Darwin در شکل ۲-۱۷ نشان داده می‌شود.

در حالی که اکثر سیستم‌عامل‌ها یک رابط فراخوانی سیستمی منفردی را به هسته فراهم می‌کنند—مثلاً از طریق کتابخانه‌ی استاندارد C در هسته‌ی سیستم‌های یونیکس و لینوکس—Darwin **دو** رابط فراخوانی سیستمی فراهم می‌کند: فراخوانی‌های سیستمی Mach (معروف به **تله‌ها**) و فراخوانی‌های سیستمی BSD (که کارکرد POSIX را فراهم می‌کنند). رابط به این فراخوانی‌های سیستمی یک مجموعه‌ی غنی از کتابخانه‌هاست که نه تنها کتابخانه‌ی استاندارد C را شامل می‌شود، بلکه کتابخانه‌هایی را نیز در بر می‌گیرد که شبکه‌گذاری، امنیت، و پشتیبانی زبان برنامه‌نویسی (که تنها چند نمونه می‌شود) را فراهم می‌کنند.



شکل ۲-۱۷ ساختار Darwin

تحت رابط فراخوانی سیستمی، Mach سرویس‌های مبنایی سیستم‌عامل را فراهم می‌کند اعم از مدیریت حافظه، زمان‌بندی CPU، و سرویس‌های ارتباط بین پردازه‌ای (IPC) همچون عبور پیغام و فراخوانی روال دور (RPC). بیشتر کارکردی که توسط Mach فراهم می‌شود از طریق

تجریدهای هسته[1] در دسترس است که شامل وظیفه‌ها (پردازه‌ی Mach)، نخ‌ها، اشیاء حافظه، و درگاه‌ها (برای IPC استفاده می‌شود) می‌باشد. به‌عنوان یک مثال، یک برنامه‌ی کاربردی ممکن است با استفاده از فراخوانی سیستمی ()fork رابط BSD POSIX یک پردازه‌ی جدیدی ایجاد کند. Mach هم به‌نوبت خود از یک تجرید هسته‌ی وظیفه برای نمایش پردازه در هسته استفاده می‌کند.

محیط هسته علاوه بر Mach و BSD یک مجموعه‌ابزار I/O برای توسعه‌ی درایور دستگاه‌ها و ماژول‌های بارشدنی پویا فراهم می‌کند (که macOS به آن‌ها تحت عنوان **گسترش‌های هسته**[2] یا kextها رجوع می‌کند).

ما در بخش ۲-۸-۳ تشریح کردیم که چگونه سربار عبور پیغام میان سرویس‌های متفاوت در حال اجرا در فضای کاربر، کارایی ریزهسته‌ها را خدشه‌دار می‌کند. Darwin برای پرداختن به این قبیل مسائل کارایی، Mach، BSD، مجموعه‌ابزار I/O، و هرگونه گسترش هسته را به‌شکل یک فضای آدرس منفردی ترکیب می‌کند. بنابراین، از این نظر یک ریزهسته‌ی محض نیست که زیرسیستم‌های مختلف در فضای کاربر اجرا می‌شوند. عبور پیغام در میان Mach همچنان اتفاق می‌افتد، اما هیچ کپی‌کردنی ضرورتی ندارد، چرا که سرویس‌ها به فضای آدرس یکسانی دسترسی دارند.

اَپل سیستم‌عامل Darwin را به‌صورت متن‌باز توزیع کرده است. در نتیجه، پروژه‌های مختلفی هستند که کارکرد اضافه‌تری به Darwin اضافه کرده‌اند، همچون سیستم پنجره‌ی X-11 و پشتیبانی سیستم‌فایل‌های اضافه‌تر. با این وجود، برعکس Darwin، رابط Cocoa، همین‌طور، سایر چارچوب‌های خصوصی اَپل که برای توسعه‌ی برنامه‌های کاربردی macOS در دسترس هستند، متن‌بسته می‌باشند.

۲-۸-۵-۲ اندروید

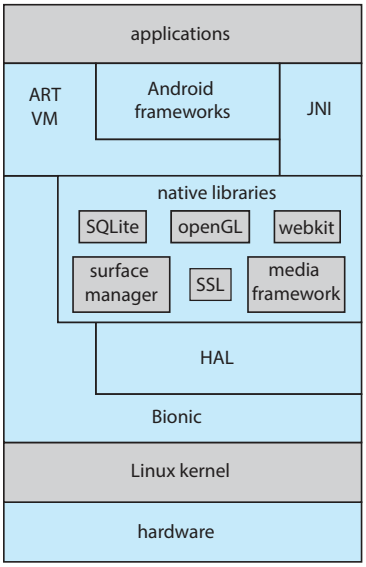
سیستم‌عامل اندروید توسط کنسرسیوم Open Handset Alliance (که در اصل، توسط گوگل رهبری می‌شود) طراحی شده و برای گوشی‌های هوشمند و کامپیوترهای تبلت نوشته شد. در حالی‌که iOS برای اجرا بر روی دستگاه‌های موبایل اپل طراحی شده و متن‌بسته است، اندروید روی انواع بسترهای موبایل اجرا شده و متن‌باز است که تا حدی، عامل رشد سریع و فراگیری آن هم می‌باشد. ساختار اندروید در شکل ۲-۱۸ مشخص است.

اندروید از این نظر مشابه iOS است که در آن، یک پشته‌ی لایه‌ای نرم‌افزار وجود دارد که یک مجموعه‌ی غنی از چارچوب‌ها را که از گرافیک، صدا، و امکانات سخت‌افزاری پشتیبانی می‌کنند فراهم می‌کند. این امکانات نیز به‌نوبت خود، سکویی برای توسعه‌ی کاربردهای موبایل که بر روی دستگاه‌های زیاد اندرویدی اجرا می‌شوند، فراهم می‌کنند.

طراحان نرم‌افزار دستگاه‌های اندروید، برنامه‌های کاربردی را به زبان جاوا می‌نویسند، اما عموماً از API استاندارد جاوا استفاده نمی‌کنند. گوگل یک API اندروید مجزا برای نوشتن جاوا طراحی کرده است. برنامه‌های جاوا به فرمی کامپایل می‌شوند که می‌توانند بر روی ART زمان

1. Kernel abstractions
2. Kernel extensions

اجرای اندروید، یک ماشین مجازی که برای اندروید طراحی شده و برای دستگاه‌های موبایل با قابلیت‌های حافظه و پردازش CPU محدود بهینه شده، اجرا شوند. برنامه‌های جاوا در ابتدا بهه یک فایل class. بایت کد جاوا کامپایل شده و سپس به یک فایل اجرایی dex. ترجمه می‌شوند. در حالی که بسیاری از ماشین‌های مجازی جاوا به‌منظور بهبود کارآمدی برنامه‌ی کاربردی کامپایل به‌موقع [1] (JIT) انجام می‌دهند، ART کامپایل **پیش از موعد** [2] (**AOT**) انجام می‌دهد. در اینجا، فایل‌های dex. موقعی که روی یک دستگاه نصب می‌شوند به کد ماشین بومی کامپایل می‌شوند که از آن می‌توانند بر روی ART اجرا شوند. کامپایل AOT اجرای کارآمدتری را در کنار کاهش مصرف برق امکان می‌دهد، ویژگی‌هایی که برای سیستم‌های موبایل حیاتی هستند.



شکل ۲-۱۸ معماری اندروید گوگل.

توسعه‌دهندگان اندروید می‌تواند برنامه‌های جاوا هم بنویسند که از رابط بومی جاوا—یا JNI—استفاده می‌کنند که به توسعه‌دهندگان اجازه می‌دهد تا ماشین مجازی را دور زده و در عوض، برنامه‌های جاوا بنویسند که می‌توانند امکانات سخت‌افزاری خاصی را دست‌یابی کنند. برنامه‌هایی که با استفاده از JNI نوشته شده‌اند عموماً از یک دستگاه سخت‌افزاری به دیگری قابلیت حمل ندارند.

مجموعه‌ی کتابخانه‌های در دسترس بومی برنامه‌های کاربردی اندروید شامل چارچوب‌هایی برای توسعه‌ی مرورگرهای وب (webkit)، پشتیبانی پایگاه‌داده (SQLite)، و پشتیبانی شبکه، همچون سوکت‌های امن (SSL) می‌باشند.

1. Just-in-time (JIT)
2. Ahead-of-time (AOT)

چون اندروید می‌تواند تقریباً بر روی تعداد نامحدودی از دستگاه‌های سخت‌افزاری اجرا شود، انتخاب گوگل، انتزاعی‌ساختن سخت‌افزار فیزیکی از طریق لایه‌ی تجرید سخت‌افزار[1]، یا HAL می‌باشد. با انتزاعی‌ساختن تمامی سخت‌افزار، همچون دوربین، تراشه‌ی GPS، و سایر حس‌گرها، HAL برای برنامه‌های کاربردی، یک دید سازگار مستقلی از سخت‌افزار خاص فراهم می‌کند. البته، این ویژگی به توسعه‌دهندگان امکان می‌دهد تا برنامه‌هایی را بنویسند که در میان سکوهای سخت‌افزاری متفاوت قابلیت حمل دارند.

کتابخانه‌ی استاندارد C استفاده شده توسط سیستم‌های لینوکس، کتابخانه‌ی C گنو (glibc) می‌باشد. گوگل در عوض، کتابخانه‌ی استاندارد C **بیونیک**[2] را برای اندروید توسعه داده است. بیونیک نه تنها ردپای حافظه‌ی کوچک‌تری به‌نسبت glibc دارد، بلکه برای CPUهای کندتری طراحی شده است که مشخصه‌ی دستگاه‌های موبایل است. (به‌علاوه، بیونیک به گوگل اجازه می‌دهد تا مجوز GPL کتابخانه‌ی glibc را دور بزند.)

در ته پشته‌ی نرم‌افزاری اندروید، هسته‌ی لینوکس است. گوگل، هسته‌ی لینوکسی را که در اندروید استفاده می‌شود در بخش‌های گوناگونی، همچون مدیریت برق اصلاح کرده است تا نیازهای خاص سیستم‌های موبایل را پشتیبانی نماید. در مدیریت و تخصیص حافظه نیز تغییراتی داده است و شکلی جدیدی از IPC تحت عنوان **پیونددهنده**[3] (که در بخش ۳-۸-۲-۱ پوشش خواهیم داد) اضافه کرده است.

زیرسیستم ویندوز برای لینوکس

ویندوز از یک معماری ترکیبی استفاده می‌کند که زیرسیستم‌هایی برای تقلید محیط‌های سیستم‌عامل متفاوت فراهم می‌کند. این زیرسیستم‌ها با هسته‌ی ویندوز ارتباط برقرار می‌کنند تا سرویس‌های واقعی را فراهم کنند. ویندوز ۱۰ یک زیرسیستم ویندوز برای لینوکس (**WSL**) اضافه می‌کند که اجازه می‌دهد تا برنامه‌های بومی لینوکس (که به‌صورت دودویی‌های ELF توصیف شده‌اند) در ویندوز ۱۰ اجرا شوند. عملیات نوعی برای یک کاربر این است که برنامه‌ی bash.exe ویندوز را اجرا کند که یک پوسته‌ی bash اجرای لینوکس برای کاربر ارائه می‌دهد. WSL در داخل یک **نمونه‌ی لینوکس** شامل پردازه‌ی init ایجاد می‌کند که به‌نوبت خود پوسته‌ی bash را ایجاد می‌کند که برنامه‌ی بومی /bin/bash لینوکس را اجرا می‌نماید. هر یک از این پردازه‌ها در یک پردازه‌ی Pico ویندوز اجرا می‌شوند. این پردازه‌ی خاص، دودویی بومی لینوکس را در فضای آدرس مال پردازه بارگذاری می‌کند، لذا یک محیطی فراهم می‌کند که در آن یک برنامه‌ی لینوکس می‌تواند اجرا شود.

پردازه‌های Pico با سرویس‌های هسته‌ی LXCore و LXSS ارتباط برقرار می‌کنند تا فراخوانی‌های سیستمی لینوکس را اگر با فراخوانی‌های سیستمی بومی ویندوز امکان‌پذیر باشند، ترجمه کنند. موقعی که برنامه‌ی لینوکس یک فراخوانی سیستمی انجام می‌دهد که معادل ویندوزی ندارد، سرویس LXSS بایستی کارکرد معادلی را فراهم نماید. موقعی که یک تناظر یک‌به‌یکی میان فراخوانی‌های سیستمی لینوکس و ویندوز وجود دارد، LXSS فراخوانی سیستمی لینوکس را مستقیماً به فراخوانی معادل آن در هسته‌ی

1. Hardware abstraction layer (HAL)
2. Bionic
3. Binder